COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE212	**SECTION:**	CpE31s2
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Jeus Guille V. Pizarro	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

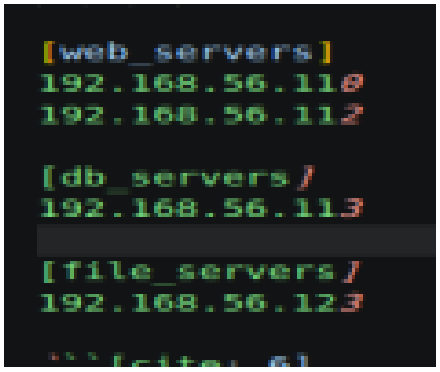
1. PROCEDURE OUTPUT

What procedures did you perform?
ask 1: Targeting Specific Nodes
1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.

```
- hosts: all
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```[cite: 2]

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:



Make sure to save the file and exit.

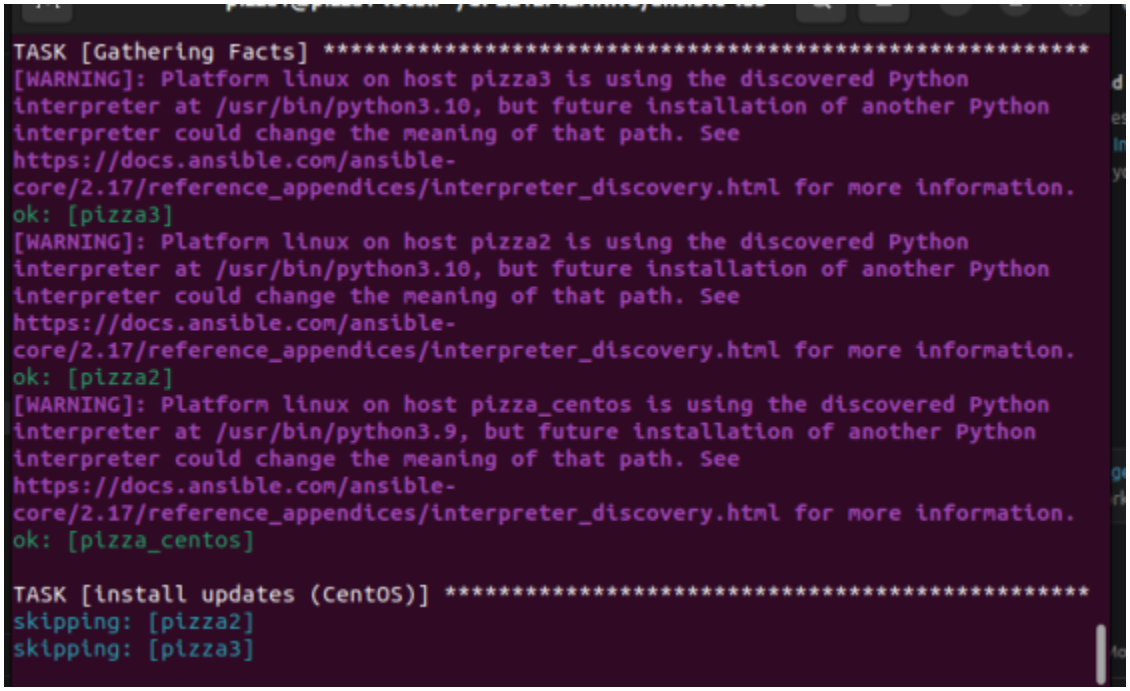
Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the site.yml by following the image below:



Make sure to save the file and exit.

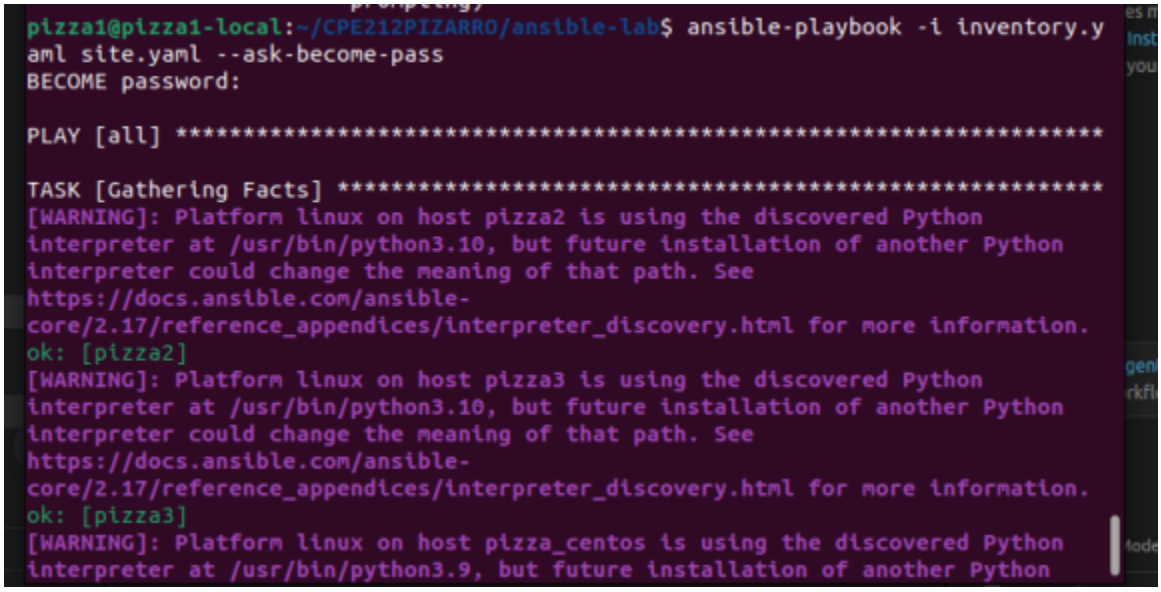
The pre-tasks command tells the ansible to run it before any other thing. In the pre-tasks, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at web_servers. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.



Run the site.yml file and describe the result.
It work

4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the playbooks from task 1.3.
Make sure to save the file and exit.
Run the site.yml file and describe the result.

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: systemctl status mariadb. Do this on the CentOS server also.



Describe the output.

6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.

Make sure to save the file and exit.

Run the site.yml file and describe the result.

The testing of the file_servers is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the site.yml file. Add tags to the playbook. After the name, we can place the tags: name_of_tag. This is an arbitrary command, which means you can use any name for a tag.

```
---
- hosts: all
  become: true
  pre_tasks:
    - name: install updates (CentOS)
      tags: always
      dnf:
        update_only: yes
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install updates (Ubuntu)
      tags: always
      apt:
        upgrade: dist
        update_cache: yes
      when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.
Run the site.yml file and describe the result.
2. On the local machine, try to issue the following commands and describe each result:
2.1 ansible-playbook --list-tags site.yml

```
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playbook --list-tags si
te.yaml

playbook: site.yaml

  play #1 (all): all    TAGS: []
      TASK TAGS: [always]

  play #2 (web_servers): web_servers    TAGS: []
      TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

  play #3 (db_servers): db_servers      TAGS: []
      TASK TAGS: [centos, db, mariadb, ubuntu]
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
```

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

```
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playbook --tags centos
--ask-become-pass  site.yaml
BECOME password:

PLAY [all] ********************************************************************

TASK [Gathering Facts] ********************************************************
[WARNING]: Platform linux on host pizza2 is using the discovered Python
interpreter at /usr/bin/python3.10, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [pizza2]
[WARNING]: Platform linux on host pizza3 is using the discovered Python
interpreter at /usr/bin/python3.10, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
ok: [pizza3]
[WARNING]: Platform linux on host pizza_centos is using the discovered Python
interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible-
core/2.17/reference_appendices/interpreter_discovery.html for more information.
```

```
TASK [install apache and php for CentOS servers] ******************************
skipping: [pizza2]
skipping: [pizza3]

PLAY [db_servers] *************************************************************

TASK [Gathering Facts] ********************************************************
ok: [pizza_centos]

TASK [install mariadb package (CentOS)] ***************************************
ok: [pizza_centos]

PLAY RECAP ********************************************************************
pizza2                     : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza3                     : ok=3    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza_centos               : ok=4    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
```

2.3 ansible-playbook --tags db --ask-become-pass site.yml

```
TASK [Gathering Facts] *********************************************************
ok: [pizza2]
ok: [pizza3]

PLAY [db_servers] **************************************************************

TASK [Gathering Facts] *********************************************************
ok: [pizza_centos]

TASK [install mariadb package (CentOS)] ****************************************
ok: [pizza_centos]

TASK [install mariadb packege (Ubuntu)] ****************************************
skipping: [pizza_centos]

PLAY RECAP *********************************************************************
pizza2                     : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
pizza3                     : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0
pizza_centos               : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
```

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

```
ok: [pizza3]

TASK [install apache and php for Ubuntu servers] *******************************
ok: [pizza2]
ok: [pizza3]

TASK [install apache and php for CentOS servers] *******************************
skipping: [pizza2]
skipping: [pizza3]

PLAY [db_servers] **************************************************************

TASK [Gathering Facts] *********************************************************
ok: [pizza_centos]

PLAY RECAP *********************************************************************
pizza2                     : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza3                     : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza_centos               : ok=3    changed=0    unreachable=0    failed=0    s
kipped=1    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ s
```

2.5 ansible-playbook --tags “apache,db” --ask-become-pass site.yml

```
TASK [install apache and php for CentOS servers] ******************************
skipping: [pizza2]
skipping: [pizza3]

PLAY [db_servers] *************************************************************

TASK [Gathering Facts] ********************************************************
ok: [pizza_centos]

TASK [install mariadb package (CentOS)] ***************************************
ok: [pizza_centos]

TASK [install mariadb packege (Ubuntu)] ***************************************
skipping: [pizza_centos]

PLAY RECAP ********************************************************************
pizza2                     : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza3                     : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0
pizza_centos               : ok=4    changed=0    unreachable=0    failed=0    s
kipped=2    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
```

Your Technological Institute of the Philippines chats aren't used to improve our models. Gemini is AI and can m

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

Figure 3.1.1

Make sure to save the file and exit.

```
  - name: start httpd (Centos)
    tags: apache, centos, httpd
    service:
      name: httpd
      state: started
      enabled: true
    when: ansible_distribution == "CentOS"
```

You would also notice from our previous activity that we already created a module that runs a service.

Figure 3.1.2

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

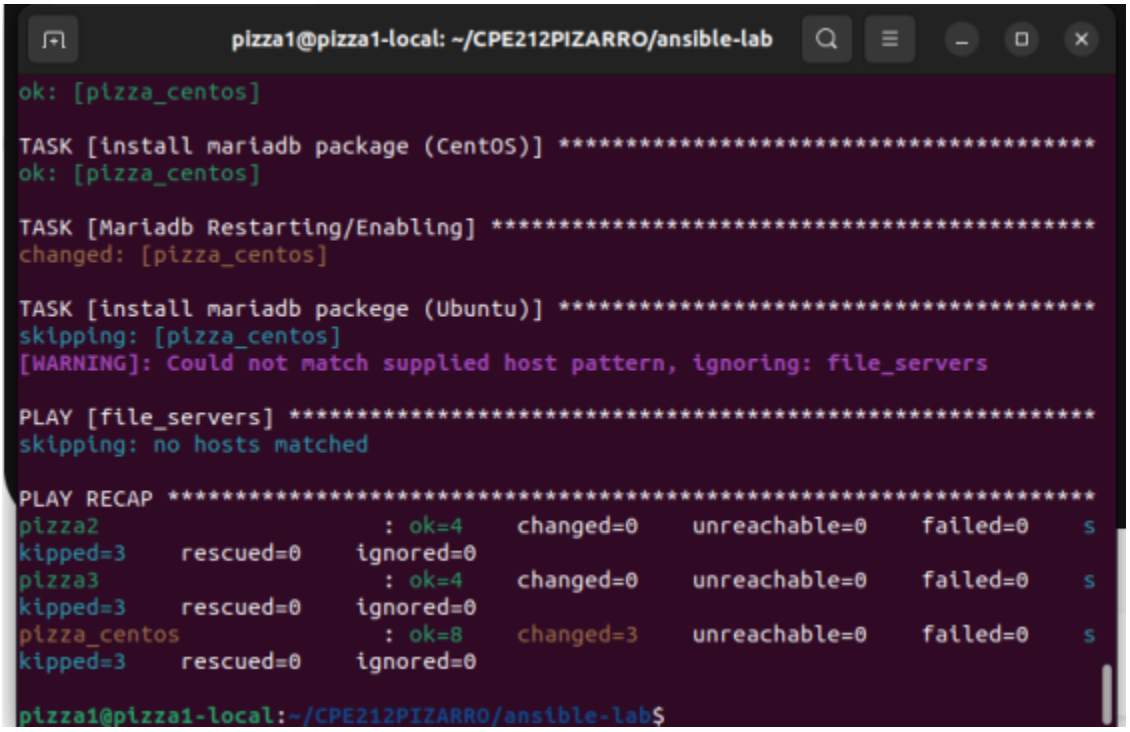
2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command sudo systemctl stop httpd. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.

```
jeus@localhost ~]$ sudo systemctl stop httpd
sudo] password for jeus:
jeus@localhost ~]$
```

3. Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

To automatically enable the service every time we run the playbook, use the command enabled:

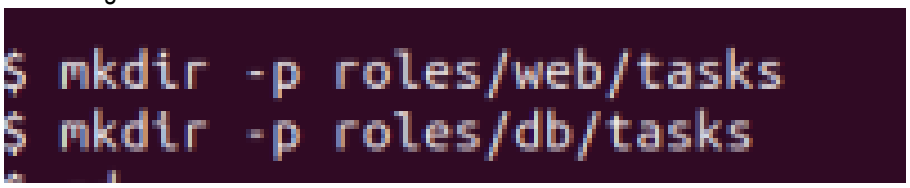
true similar to Figure 7.1.2 and save the playbook



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. Creating a Role for Web Servers
2. Creating a Role for Web Servers
3. Creating a Role for Database Servers



4. Combining Multiple Roles in a Playbook

```yaml
---
- hosts: web_servers
  become: true
  roles:
    - role: web
      tags: web

- hosts: db_servers
  become: true
  roles:
    - role: db
      tags: db
```

```yaml
---
- name: install mariadb package (CentOS)
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

- name: Mariadb Restarting/Enabling
  service:
    name: mariadb
    state: restarted
    enabled: true

- name: install mariadb package (Ubuntu)
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"
```

To orchestrate multiple roles in a single playbook for different server groups.
Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for Database.

```
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
ook site2.yml -K --tags db

pizza2                     : ok=1    changed=0    unreachable=
0    failed=0    skipped=0    rescued=0    ignored=0
pizza3                     : ok=1    changed=0    unreachable=
0    failed=0    skipped=0    rescued=0    ignored=0
pizza_centos               : ok=4    changed=1    unreachable=
0    failed=0    skipped=1    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
```

```
pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$ ansible-playb
ook site2.yml -K --tags web

pizza2                     : ok=2    changed=0    unreachable=
0    failed=0    skipped=2    rescued=0    ignored=0
pizza3                     : ok=2    changed=0    unreachable=
0    failed=0    skipped=2    rescued=0    ignored=0
pizza_centos               : ok=4    changed=0    unreachable=
0    failed=0    skipped=1    rescued=0    ignored=0

pizza1@pizza1-local:~/CPE212PIZARRO/ansible-lab$
```

Reflections:
Through this lab activity, I gained hands-on experience in structuring Ansible automation for multi-OS environments containing Ubuntu and CentOS nodes. I learned how inventory grouping and condition-based task execution prevent configuration drift while ensuring software is deployed only to intended hosts. Transitioning from a single playbook to a modular, role-based architecture (roles/web and roles/db) demonstrated how to write clean, reusable automation code that simplifies long-term infrastructure management

Answer the following:

1. What is the importance of putting our remote servers into groups?
Grouping remote servers allows targeted task execution based on functional roles such as web, database, or file servers. This prevents deploying unnecessary software across unrelated nodes and streamlines configuration management across heterogeneous operating systems.

2. What is the importance of tags in playbooks?Tags assign metadata to specific plays, tasks, or roles so administrators can execute targeted portions of a playbook. This significantly reduces execution time during minor updates and isolates troubleshooting without re-running the entire workflow.

3. Why do you think some services need to be managed automatically in playbooks?
Automated service management guarantees that installed packages start immediately and remain enabled across server reboots. It also enforces self-healing infrastructure by detecting stopped services and automatically restoring them to an operational state.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **quilbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.